



CODING / DSA

By Abhishek Rathor

Instagram: code.abhii07 (SYNTAX ERROR)

☒ Top 50 Coding Interview Questions

◆ 1. Print Array

```
{
// start loop from first index
for i = 0 to n-1
{
// print current element
print a[i]
}
}
```

◆ 2. Reverse Array

```
{
// point to first index
i = 0

// point to last index
j = n - 1

// loop until both meet
while i < j
{
// swap elements
swap a[i] and a[j]

// move forward
i++

// move backward
j--
}
}
```

◆ 3. Find Largest Number

```
{
// assume first element is maximum
max = a[0]
```

```
// traverse remaining elements
for i = 1 to n-1
{
    // compare current with max
    if a[i] > max
    {
        // update max value
        max = a[i]
    }
}
}
```

◆ 4. Linear Search

```
{
    // traverse entire array
    for i = 0 to n-1
    {
        // check if element matches key
        if a[i] == key
        {
            // element found
            print "Found"
        }
    }
}
```

◆ 5. Sum of Array

```
{
    // initialize sum as zero
    sum = 0

    // loop through array
    for i = 0 to n-1
    {
        // add each element to sum
        sum = sum + a[i]
    }
}
```

◆ 6. Even or Odd Number

```
{
    // check remainder when divided by 2
    if n % 2 == 0
    {
        // number is even
        print "Even"
    }
    else
    {

```

```
// number is odd
print "Odd"
}
}
```

◆ 7. Factorial of Number

```
{
// initialize factorial value
fact = 1

// loop from 1 to n
for i = 1 to n
{
// multiply current value
fact = fact * i
}
}
```

◆ 8. Reverse Number

```
{
// store reversed number
rev = 0

// repeat until number becomes zero
while n > 0
{
// take last digit and add
rev = rev * 10 + n % 10

// remove last digit
n = n / 10
}
}
```

◆ 9. Palindrome Number

```
{
// compare original and reversed number
if original == reverse
{
// number is palindrome
print "Palindrome"
}
else
{
// number is not palindrome
print "Not Palindrome"
}
}
```

◆ 10. Fibonacci Series

```
{
// first number
a = 0

// second number
b = 1

// loop n times
for i = 1 to n
{
// print current number
print a

// calculate next number
c = a + b

// update first value
a = b

// update second value
b = c
}
}
```

◆ 11. Count Digits

```
{
// initialize digit counter
count = 0

// loop until number becomes zero
while n > 0
{
// increase digit count
count++

// remove last digit
n = n / 10
}
}
```

◆ 12. Sum of Digits

```
{
// initialize sum
sum = 0

// repeat until number is zero
while n > 0
{
// add last digit to sum
sum = sum + n % 10
}
```

```
// remove last digit
n = n / 10
}
}
```

◆ 13. Prime Number Check

```
{
// assume number is prime
flag = true

// loop from 2 to n-1
for i = 2 to n-1
{
// check divisibility
if n % i == 0
{
// number is not prime
flag = false
}
}
}
```

◆ 14. Swap Two Numbers

```
{
// store first value
temp = a

// copy second into first
a = b

// copy temp into second
b = temp
}
```

◆ 15. Bubble Sort

```
{
// outer loop for passes
for i = 0 to n-1
{
// inner loop for comparison
for j = 0 to n-i-2
{
// compare adjacent elements
if a[j] > a[j+1]
{
// swap elements
swap a[j] and a[j+1]
}
}
}
```

```
}  
}
```

◆ 16. Find Smallest Element

```
{  
  // assume first element is smallest  
  min = a[0]  
  
  // check remaining elements  
  for i = 1 to n-1  
  {  
    // compare with min  
    if a[i] < min  
    {  
      // update min value  
      min = a[i]  
    }  
  }  
}
```

◆ 17. Reverse String

```
{  
  // start index  
  i = 0  
  
  // last index  
  j = length - 1  
  
  // loop until indices meet  
  while i < j  
  {  
    // swap characters  
    swap s[i] and s[j]  
  
    // move forward  
    i++  
  
    // move backward  
    j--  
  }  
}
```

◆ 18. Count Vowels

```
{  
  // initialize count  
  count = 0  
  
  // loop through string  
  for each character in string  
  {
```

```
// check vowel
if character is a,e,i,o,u
{
    // increase count
    count++
}
}
```

◆ 19. Stack Push

```
{
    // move top to next position
    top = top + 1

    // insert value
    stack[top] = value
}
```

◆ 20. Stack Pop

```
{
    // take top value
    value = stack[top]

    // decrease top
    top = top - 1
}
```

◆ 21. Queue Enqueue

```
{
    // move rear forward
    rear = rear + 1

    // insert value
    queue[rear] = value
}
```

◆ 22. Queue Dequeue

```
{
    // move front forward
    front = front + 1
}
```

◆ 23. Linked List Traversal

```
{
    // start from head
    ptr = head

    // loop till end
    while ptr != NULL
    {
        // print data
        print ptr->data

        // move to next node
        ptr = ptr->next
    }
}
```

◆ 24. Insert at Beginning (Linked List)

```
{
    // point new node to head
    new->next = head

    // update head
    head = new
}
```

◆ 25. Delete First Node (Linked List)

```
{
    // store head node
    temp = head

    // move head to next node
    head = head->next

    // delete old head
    delete temp
}
```

◆ 26. Binary Search

```
{
    // set starting index
    low = 0

    // set ending index
    high = n - 1

    // loop until range exists
    while low <= high
    {
        // find middle index
        mid = (low + high) / 2
    }
}
```

```
}
```

◆ 27. GCD (Euclid Method)

```
{  
  // repeat until remainder is zero  
  while b != 0  
  {  
    // find remainder  
    r = a % b  
  
    // shift values  
    a = b  
    b = r  
  }  
}
```

◆ 28. LCM

```
{  
  // calculate LCM using GCD  
  lcm = (a * b) / gcd  
}
```

◆ 29. Find Missing Number

```
{  
  // calculate total sum  
  total = n * (n + 1) / 2  
  
  // subtract array sum  
  missing = total - arraySum  
}
```

◆ 30. Check Sorted Array

```
{  
  // loop through array  
  for i = 0 to n-2  
  {  
    // check order  
    if a[i] > a[i+1]  
    {  
      // array is not sorted  
      print "Not Sorted"  
    }  
  }  
}
```

◆ 31. Find Second Largest Element

```
{
// assume first two values
largest = -1
second = -1

// loop through array
for i = 0 to n-1
{
// if current element is greater than largest
if a[i] > largest
{
// update second largest
second = largest

// update largest
largest = a[i]
}
}
}
```

◆ 32. Count Even Numbers

```
{
// initialize counter
count = 0

// traverse array
for i = 0 to n-1
{
// check even number
if a[i] % 2 == 0
{
// increase count
count++
}
}
}
```

◆ 33. Count Odd Numbers

```
{
// initialize counter
count = 0

// traverse array
for i = 0 to n-1
{
// check odd number
if a[i] % 2 != 0
{
// increase count
count++
}
}
```

```
}  
}  
}
```

◆ 34. Matrix Addition

```
{  
  // loop through rows  
  for i = 0 to r-1  
  {  
    // loop through columns  
    for j = 0 to c-1  
    {  
      // add corresponding elements  
      c[i][j] = a[i][j] + b[i][j]  
    }  
  }  
}
```

◆ 35. Matrix Transpose

```
{  
  // loop through rows  
  for i = 0 to r-1  
  {  
    // loop through columns  
    for j = 0 to c-1  
    {  
      // swap rows and columns  
      t[j][i] = a[i][j]  
    }  
  }  
}
```

◆ 36. Power of a Number

```
{  
  // initialize result  
  result = 1  
  
  // repeat p times  
  for i = 1 to p  
  {  
    // multiply number  
    result = result * n  
  }  
}
```

◆ 37. Armstrong Number

```
{
// initialize sum
sum = 0

// store original number
temp = n

// loop until number becomes zero
while temp > 0
{
// take last digit
digit = temp % 10

// add cube of digit
sum = sum + digit * digit * digit

// remove last digit
temp = temp / 10
}
}
```

◆ 38. Count Frequency of Elements

```
{
// loop through array
for i = 0 to n-1
{
// count occurrences of a[i]
frequency[a[i]]++
}
}
```

◆ 39. Remove Duplicates

```
{
// loop through array
for i = 0 to n-1
{
// check if element already exists
if a[i] not in newArray
{
// insert unique element
insert a[i] into newArray
}
}
}
```

◆ 40. Two Sum Problem

```
{
// check all pairs
for i = 0 to n-1
{
```

```
for j = i+1 to n-1
{
    // check required sum
    if a[i] + a[j] == target
    {
        print "Pair Found"
    }
}
}
```

◆ 41. Reverse Linked List

```
{
    // initialize pointers
    prev = NULL
    curr = head

    // loop through list
    while curr != NULL
    {
        // store next node
        next = curr->next

        // reverse link
        curr->next = prev

        // move pointers
        prev = curr
        curr = next
    }

    // update head
    head = prev
}
```

◆ 42. Tree Inorder Traversal

```
{
    // if node exists
    if root != NULL
    {
        // visit left subtree
        inorder(root->left)

        // print root value
        print root->data

        // visit right subtree
        inorder(root->right)
    }
}
```

◆ 43. Tree Preorder Traversal

```
{
// if node exists
if root != NULL
{
// print root value
print root->data

// visit left subtree
preorder(root->left)

// visit right subtree
preorder(root->right)
}
}
```

◆ 44. Tree Postorder Traversal

```
{
// if node exists
if root != NULL
{
// visit left subtree
postorder(root->left)

// visit right subtree
postorder(root->right)

// print root value
print root->data
}
}
```

◆ 45. Depth First Search (DFS)

```
{
// mark node as visited
visited[node] = true

// visit adjacent nodes
for each neighbour
{
// if not visited
if not visited
{
DFS(neighbour)
}
}
}
```

◆ 46. Breadth First Search (BFS)

```

{
    // insert starting node in queue
    enqueue(start)

    // mark start visited
    visited[start] = true

    // loop until queue empty
    while queue not empty
    {
        // remove node
        node = dequeue()

        // visit neighbours
        for each neighbour
        {
            // if not visited
            if not visited
            {
                enqueue(neighbour)
                visited[neighbour] = true
            }
        }
    }
}

```

◆ 47. Check Balanced Parentheses

```

{
    // loop through expression
    for each character
    {
        // push opening bracket
        if opening bracket
            push stack

        // pop for closing bracket
        else if closing bracket
            pop stack
    }
}

```

◆ 48. Kadane's Algorithm

```

{
    // initialize values
    currentSum = 0
    maxSum = 0

    // loop through array
    for i = 0 to n-1
    {
        // add current element
        currentSum = currentSum + a[i]

        // update max sum
    }
}

```

```
if currentSum > maxSum
    maxSum = currentSum

// reset if negative
if currentSum < 0
    currentSum = 0
}
```

◆ 49. Find Duplicate Element

```
{
// compare each element
for i = 0 to n-1
{
    for j = i+1 to n-1
    {
        // check duplicate
        if a[i] == a[j]
            print "Duplicate Found"
    }
}
}
```

◆ 50. Count Words in String

```
{
// initialize count
count = 1

// loop through string
for each character
{
    // count spaces
    if character == space
        count++
}
}
```

Connect & Share:

Tag us on your success stories:

- Instagram: [@code.abhii07](#)
- YouTube: [SYNTAX ERROR](#)